



VulGuard: Vulnerability Detection and Resolution Tool

CSC-300 Lab Based Project (LBP)

Aadit Kumar Sahoo 23114001

Utkarsh Kumar 23114101

May 2026

Department of Computer Science and Engineering

Indian Institute of Technology Roorkee

Supervisor: Dr. Sandeep Kumar Garg





- 1 Introduction
- 2 Motivation
- 3 Initial Work
- 4 Related Work
- 5 Proposed Method
- 6 Experiment Evaluation
- 7 Conclusion
- 8 References





- ❑ Software vulnerabilities are a **critical and growing** risk in modern applications
- ❑ Complexity compounds the problem: third-party components, multi-environment deployments, interconnected supply chains
- ❑ A single flaw can propagate to *all linked systems* via the supply chain
- ❑ Organizations require **automated, scalable detection** to minimise risk and remediation cost

Core Tension

Large codebases and fast release cadences **outpace** manual auditing. Tools must balance **speed**, **precision**, and **developer usability**.

The Gap

Existing tools either **lack accuracy** (static analysis) or **lack integration** (ML prototypes) forcing developers out of their workflow.

Shortcomings of Current Tools



Static Analysis

High false-positive rates from lack of code-context understanding $\Rightarrow$ *alert fatigue* and delayed remediation

ML Research Prototypes

State-of-the-art accuracy but depend on custom operating setups; outputs are difficult for practitioners to interpret

Dynamic Analysis / Fuzzing

Requires runtime environments, complex setup, and substantial compute impractical for every code change

IDE Integration Gap

No seamless in-editor workflow developers must export code, run external tools, and parse logs manually

Key Insight

Effective vulnerability detection must be **explainable**, **IDE-native**, and **low-latency** to be adopted in real development practice.



- **Shallow feature extraction:** Lexical / heuristic tools miss semantic, control-flow, and data-flow vulnerabilities; false alerts fire on safe code
- **Noisy, heterogeneous codebases:** Models trained on controlled datasets degrade on real-world legacy and mixed code
- **Fragmented workflows:** Prototypes run outside the IDE, requiring context switching and manual log interpretation
- **Coarse-grained outputs:** Function-level alerts without line localisation slow triage and code review
- **Weak explanations & fixes:** Scores without reasoning; patches that ignore surrounding code semantics
- **Latency constraints:** Heavy cold-start pipelines block iterative development; teams need fast, repeatable analysis at scale

Initial Work: Vulnerability Scoring Systems



Scoring System Analysis

- ❑ Studied **CVSS**, **VRSS**, **WIVSS**, and the hybrid **VIEWSS** model
- ❑ Fixed Impact:Exploitability ratios can undervalue high-impact, hard-to-exploit flaws
- ❑ VIEWSS uses qualitative CIA-weighting to select among **6:4**, **5:5**, or **4:6** distributions

Empirical Evaluation

- ❑ Assessed **CVSS**, **SSVC**, **EPSS**, and Exploitability Index on Patch Tuesday disclosures
- ❑ Systems frequently disagree; weak cross-system correlation
- ❑ EPSS struggles to predict exploitation prior to public catalog confirmation

Outcomes

Scoring treated as a **flexible, multi-signal system** with transparent computation. Preserved as a well-defined direction for **future work**.



- ❑ **VulExplainer** [8] Transformer-based hierarchical distillation for human-readable vulnerability type explanations; locally verified on test cases.
- ❑ **VulRepair** [9] T5-based sequence-to-sequence repair; demonstrates feasibility of automated patch generation from vulnerable C/C++ code.
- ❑ **VulDeePecker** [2] Deep learning over code gadgets; underscores the importance of high-quality training data for effective detection.
- ❑ **GRACE** [7] Graph neural network with context fusion; surpasses local sequence-based detection by integrating multi-level contextual signals.



- ❑ **Russell et al.** [3] Automated deep feature extraction at function level; outperforms manual rule-based systems on noisy real-world code.
- ❑ **SySeVR** [4] Combines syntax patterns with semantic relationships, enabling detection beyond surface-level token recognition.
- ❑ **REVEAL** [5] Graph neural networks over code property graphs; captures complex data and control-flow dependencies.
- ❑ **Devign** [6] Gated graph neural networks over structured program representations; structured analysis enhances learned embeddings.
- ❑ **LineVul** [1] Line-level vulnerability detection directly motivating VulGuard's inline highlighting and targeted patch generation.



Core Design

IDE-integrated security tool for C/C++ combining **deep learning inference**, **retrieval-augmented generation (RAG)**, and a **review-first patch workflow** inside Visual Studio Code.

Two Primary Components:

- ❑ **VS Code Extension (Front-end)**
Captures active buffer, dispatches to inference server, renders inline highlights, manages user interaction
- ❑ **Python Inference Server (Back-end)** Executes line-level vulnerability model;

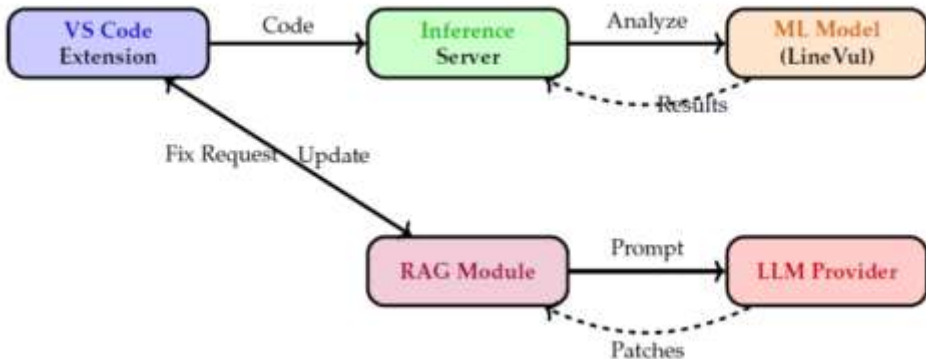
Communication:

- ❑ Lightweight **JSON** protocol
- ❑ File path + **SHA-256** hash → structured response
- ❑ Server handles resource-intensive inference
- ❑ Extension remains **responsive** at all times

Design Principle

Separation of concerns: heavy ML inference never blocks the IDE's user interface.

System Architecture



VulGuard System Architecture: Vulnerability Detection and RAG-based Fix Generation Pipeline



1. Input Capture & Preprocessing

- ☐ Reads active editor buffer; normalises line endings to a canonical form
- ☐ Computes SHA-256 content hash as a revision token for snapshot integrity

2. Model Inference

- ☐ Snapshot serialised as JSON and dispatched to the inference server
- ☐ Server performs tokenisation + model forward pass (LineVul backbone)
- ☐ Returns **per-line vulnerability scores** and a ranked candidate list

3. Evidence Enrichment

- ☐ Contextual window constructed around each flagged candidate line
- ☐ Evidence snippets scored by proximity and semantic relevance
- ☐ Provides grounding material for the downstream LLM generation stage



4. Explanation & Fix Generation

- ❑ Structured prompt assembled from findings, evidence snippets, and an output schema constraint
- ❑ LLM generates per-finding **explanations** and multiple ranked **patch candidates**
- ❑ Supports local **Ollama** instances or configurable cloud LLM endpoints

5. Review & Application

- ❑ Ranked patch candidates presented **in-editor** for human evaluation
- ❑ **Explicit user consent required** before any modification is applied
- ❑ Patch applied atomically via editor edit API; one-click re-analysis available post-patch

Performance Note

Incremental phase execution with **cached artefacts** reduces cold-start overhead and enables resumable analysis across edit sessions.

Retrieval-Augmented Generation & Fix Suggestion



Purpose: Ground LLM patch generation in local code semantics to suppress hallucination and maintain surrounding code intent.

Process:

- ❑ Bounded context window around each vulnerable line (configurable token budget)
- ❑ Evidence snippets **deduplicated** and ranked by closeness and relevance
- ❑ Multiple patch candidates returned and ranked

Patch Ranking Criteria:

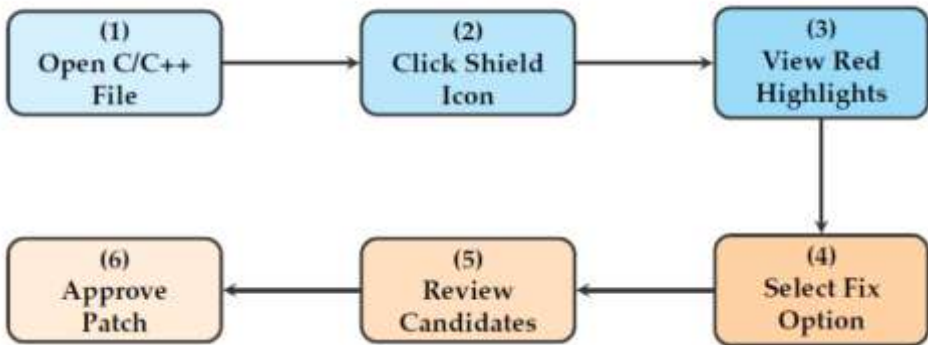
- ❑ Minimal diff size (smallest safe change preferred)
- ❑ Syntax validity verification
- ❑ Semantic consistency with surrounding code context

Patch Quality RAG vs. Alternatives

Variant	Syn.	Apply	Acc.
Template fixes	61%	52%	38%
LLM only	79%	63%	49%
RAG+LLM	88%	76%	61%

RAG yields **+23 pp** developer acceptance vs. templates.

User Workflow: Six-Step Interaction



VulGuard User Workflow: Six-Step Process for Vulnerability Detection and Resolution

Typical Latency

Detection: ~**2–10 s** Fix application:
additional **6–8 s**

Human-in-the-Loop

No silent code changes explicit user approval required before every patch.



Evaluation Metrics:

Metric	Definition
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$
Precision	$TP / (TP + FP)$
Recall	$TP / (TP + FN)$
F1-Score	$2 \cdot \text{Prec} \cdot \text{Rec} / (\text{Prec} + \text{Rec})$
Localization@k	Fraction of vulnerabilities with vuln. line in top-k output

Datasets (C/C++ functions, confirmed vulnerability labels):

Dataset	Functions	Vuln.%	Avg. Lines	Files
FFmpeg+Qemu	21,400	5.8	94	5,200
Big-Vul	34,900	7.2	78	12,600

Baselines

SySeVR · VulDeePecker · Russell et al. ·
Deign · REVEAL · LineVul · GRACE

Results: FFmpeg+Qemu Dataset



Method	Acc	Prec	Rec	F1
SySeVR ^[4]	47.85	46.06	58.81	51.66
VulDeePecker ^[2]	49.91	46.05	32.55	38.14
Russell et al. ^[3]	57.60	54.76	40.72	46.71
Devign ^[6]	56.89	52.50	64.67	57.95
REVEAL ^[5]	61.07	55.50	70.70	62.19
LineVul ^[1]	63.87	63.43	47.57	54.37
GRACE ^[7]	59.78	53.94	82.13	65.11
VulGuard	63.43	56.93	81.38	67.00

- ❑ VulGuard achieves the **highest F1 (67.00)** on FFmpeg+Qemu
- ❑ Outperforms next-best GRACE (65.11) while maintaining competitive precision
- ❑ Strong recall (**81.38**) fewer missed vulnerabilities

Results: Big-Vul Dataset



Method	Acc	Prec	Rec	F1
SySeVR ^[4]	90.10	30.91	14.08	19.34
VulDeePecker ^[2]	81.19	38.44	12.75	19.15
Russell et al. ^[3]	86.85	14.86	26.97	19.17
Devign ^[6]	92.78	30.61	15.96	20.98
REVEAL ^[5]	87.14	17.22	34.04	22.87
LineVul ^[1]	99.02	95.13	87.01	90.89
GRACE ^[7]	90.73	32.52	39.08	35.50
VulGuard	95.17	95.37	87.30	91.95

- ❑ VulGuard achieves the **highest F1 (91.95)** on Big-Vul
- ❑ Surpasses LineVul (90.89) with marginally higher precision (95.37 vs. 95.13)
- ❑ Demonstrates strong generalisation across both large-scale datasets



Line-Level Localisation Accuracy: [?]

Dataset	Top-1 (%)	Top-3 (%)	Line-F1 (%)
FFmpeg+Qemu	62.4	79.1	71.5
Big-Vul	74.8	90.6	83.2

Strong Top-3 accuracy means developers reach the vulnerable line with **minimal scrolling**.

Patch Generation Quality RAG vs. Alternatives:

Variant	Syntax Valid (%)	Applies Cleanly (%)	Accepted (%)
Template fixes	61	52	38
LLM only	79	63	49
RAG + LLM	88	76	61

Contextual grounding via RAG yields the largest gain in developer acceptance (+23 pp vs. templates).



- ❑ End-to-end latency measured from editor trigger to result rendering
- ❑ Vulnerability detection: ≈ 10 s on a 2000-line file; fix application: additional 6–8 s

File Size (Lines)	Detection Only (s)	Full Pipeline (s)
200	2.1	8.5
500	4.3	11.2
1000	7.2	15.0
2000	10.0	17.5
4000	14.5	22.8

Persistent Server Mode

Inference server remains **warm** across analysis requests; retry and timeout mechanisms prevent IDE unresponsiveness. Latency scales approximately linearly with file size, remaining practical for files up to 4 000 lines.



What We Built:

- ❑ IDE-native **line-level** vulnerability detection for C/C++ in VS Code
- ❑ RAG-grounded explanation and patch generation with mandatory human review
- ❑ Separate persistent inference server heavy ML never blocks the editor

Key Contribution

A usable, explainable, and IDE-native security tool that preserves **developer autonomy** through mandatory human-in-the-loop patch approval.

Key Results:

- ❑ **F1 = 67.00** on FFmpeg+Qemu
- ❑ **F1 = 91.95** on Big-Vul
- ❑ Strong recall with low false-alarm rates
Top-3 localisation: **79.1%/ 90.6%**
- ❑ Patch acceptance: **61%** (vs. 38% for template-only fixes)
- ❑ Bridges the gap between research-level vulnerability models and everyday software engineering practice

Impact

Bridges the gap between **research-level** vulnerability models and **everyday** software engineering practice.



- ❑ **Broader language support:** Extend beyond C/C++ via multilingual models and language-specific ASTs (Python, Java, Rust, ...)
- ❑ **Patch verification:** Lightweight symbolic checks and automated unit-test replay in a sandboxed environment to validate patch safety before presentation
- ❑ **Feedback-driven learning:** Incorporate approve/reject signals to refine ranking heuristics and update prompt templates to project coding norms
- ❑ **CI/CD integration:** Headless mode for pull-request monitoring; automated security gating in early-stage development via VCS hooks
- ❑ **Scalable evidence retrieval:** Project-wide code search and dependency indexing for cross-file vulnerability context in the RAG module
- ❑ **Multi-signal scoring:** Integrate VIEWSS-style flexible scoring layer for risk prioritisation^[?]



- 1 . Zhong, H., & Wang, C. (2022). LineVul: A Deep Learning-based Line-level Vulnerability Detection Model. *ICSE*.
- 2 . Li, Z., et al. (2018). VulDeePecker: A Deep Learning-based System for Vulnerability Detection. *NDSS*.
- 3 . Russell, R., et al. (2018). Automated Vulnerability Detection in Source Code Using Deep Representation Learning. *arXiv preprint*.
- 4 . Li, Z., et al. (2019). SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE TSE*.
- 5 . Chakraborty, S., Krishna, R., Ding, H., & Ray, B. (2021). REVEAL: A Neural Network Based Vulnerability Detection System. *ACM CCS*.
- 6 . Zhou, Y., et al. (2019). Devign: Effective Vulnerability Identification by Learning Code Property Graphs. *NeurIPS*.
- 7 . Wu, J., Tan, H., & Wang, Y. (2021). GRACE: A Graph Neural Network based Framework for Vulnerability Detection. *ASE*.
- 8 . Fu, M., et al. (2023). VulExplainer: A Transformer-Based Hierarchical Distillation for Explaining Vulnerability Types. *IEEE TSE*, 49(10).
- 9 . Fu, M., et al. (2022). VulRepair: A T5-Based Automated Software Vulnerability Repair. *ESEC/FSE*.
- 10 . Sharma, A., Sabharwal, S., & Nagpal, S. (2023). A hybrid scoring system for prioritization of software vulnerabilities. *Computers & Security*, 129, 103256.
- 11 . Koscinski, V., et al. (2025). Conflicting Scores, Confusing Signals: An Empirical Study of Vulnerability Scoring Systems. *arXiv:2508.13644*.

Thank You

Questions & Discussion

Aadit Kumar Sahoo aadit_ks@cs.iitr.ac.in

Utkarsh Kumar utkarsh_k@cs.iitr.ac.in

Supervisor: Dr. Sandeep Kumar Garg
Department of Computer Science and Engineering
Indian Institute of Technology Roorkee